Neural Networks 187 (2025) 107291

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Dual-view graph-of-graph representation learning with graph Transformer for graph-level anomaly detection

Wangyu Jin [a], Huifang Ma [a,*], Yingyue Zhang [a], Zhixin Li [b], Liang Chang [c]

[a] College of Computer Science and Engineering, Northwest Normal University, Lanzhou, Gansu 730070, China
[b] Key Lab of Education Blockchain and Intelligent Technology, Ministry of Education, Guangxi Normal University, Guilin, Guangxi 541004, China
[c] School of Computer Science and Information Security, Guilin University of Electronic Technology, Guilin, Guangxi 541004, China

ARTICLE INFO

Keywords:
Graph-level anomaly detection
Graph transformer
Graph neural networks
Unsupervised graph representation learning

ABSTRACT

Graph-Level Anomaly Detection (GLAD) endeavors to pinpoint a small subset of anomalous graphs that deviate from the normal data distribution within a given set of graph data. Existing GLAD methods typically rely on Graph Neural Networks (GNNs) to extract graph-level representations, which are then used for the detection task. However, the inherent limited receptive field of GNNs may exclude crucial anomalous information embedded within the graph. Moreover, the inadequate modeling of cross-graph relationships limits the exploration of connections between different graphs, thus restricting the model's ability to uncover inter-graph anomalous patterns. In this paper, we propose a novel approach called Dual-View Graph-of-Graph Representation Learning Network for unsupervised GLAD, which takes into account both intra-graph and inter-graph perspectives. Firstly, to enhance the capability of mining intra-graph information, we introduce a Graph Transformer that enhances the receptive field of the GNNs by considering both attribute and structural information. This augmentation enables a comprehensive exploration of the information encoded within the graph. Secondly, to explicitly capture the cross-graph dependencies, we devise a Graph-of-Graph-based dual-view representation learning network to explicitly capture cross-graph interdependencies. Attribute and structure-based graph-of-graph representations are induced, facilitating a comprehensive understanding of the relationships between graphs. Finally, we utilize anomaly scores from different perspectives to quantify the extent of anomalies present in each graph. This multi-perspective evaluation provides a more comprehensive assessment of anomalies within the graph data. Extensive experiments conducted on multiple benchmark datasets demonstrate the effectiveness of our proposed method in detecting anomalies within graph data.

1. Introduction

Graphs possess unique advantages in representing complex relational data, making them an indispensable tool in various fields. In social network analysis (Jiang et al., 2024; Xu et al., 2020), graphs depict relationships between individuals, with nodes representing individuals and edges symbolizing connections such as friendships, follows, or shared interests. Similarly, in financial fraud analysis (Liu et al., 2018), graphs showcase transaction histories and fund flows among different accounts. Additionally, in bioinformatics research (Gao et al., 2023; Wang et al., 2021; Wu et al., 2024), graphs describe chemical molecules, where nodes represent atoms and edges signify chemical bonds. Consequently, in recent years, diverse graph data analysis tasks in these domains have sparked widespread research interest. Among these tasks, Graph-level Anomaly Detection (GLAD) stands out, aiming to identify a small number of anomalous graphs that deviate from the majority of normal data distributions within a group of graph data. Due to its broad applicability in security, GLAD has garnered significant attention. For instance, GLAD models can identify potential security risks in social networks (Kreuzer et al., 2021), detect fraudulent activities in financial transaction networks (Chai et al., 2023; Dwivedi & Bresson, 2021; Li et al., 2023b), or pinpoint toxic and harmful chemical molecules (Xu et al., 2019).

In recent years, significant efforts have been dedicated to GLAD research, focusing on encoding graph structure information and attribute data into graph embeddings for subsequent analysis and detection within the representation space. Following this paradigm, certain

* Corresponding author.
E-mail addresses: 2022222237@nwnu.edu.cn (W. Jin), mahuifang@yeah.net (H. Ma), 2022212152@nwnu.edu.cn (Y. Zhang), lizx@gxnu.edu.cn (Z. Li), changl@guet.edu.cn (L. Chang).

https://doi.org/10.1016/j.neunet.2025.107291
Received 6 August 2024; Received in revised form 6 January 2025; Accepted 14 February 2025
Available online 25 February 2025
0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

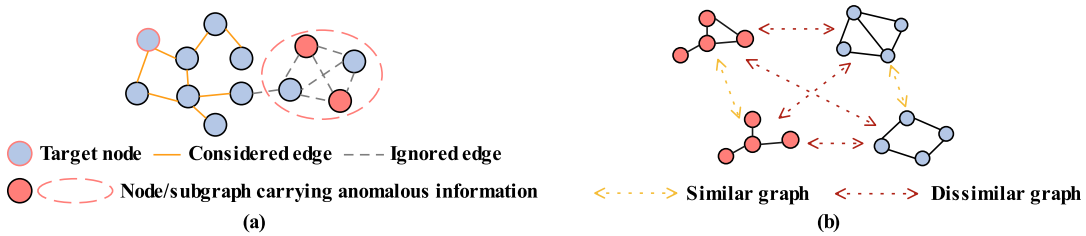


Fig. 1. (a) A toy example of the limited receptive field of GNNs. Focusing on the representation learning process of the target node, the limited receptive field of GNNs prevents the capture of information from anomalous nodes or subgraphs that are far from the target node within the same graph. (b) A toy example of inter-graph relationships. The inherent similarities or dissimilarities between graph samples can provide additional insights for detecting anomalies within the samples, aiding in the identification of a more accurate decision boundary.

two-stage methodologies utilize graph kernel functions, such as the Weisfeiler–Lehman kernel (Shervashidze et al., 2011) and the propagation kernel (Neumann et al., 2016), for graph representation learning, followed by the application of traditional machine learning techniques, e.g. Isolation Forest (Liu et al., 2008) and LOF (Breunig et al., 2000), for anomaly detection. By decoupling these two stages, the representation learner acquires informative graph-level representations, upon which the detector relies for high-quality anomaly detection. However, challenges persist for such methods due to their reliance on anomalous labels and the absence of interpretable anomaly scores. Recent advancements in Graph Neural Networks (GNNs) have catalyzed the emergence of end-to-end methodologies, seamlessly integrating representation learning and anomaly detection scores within a unified framework, effectively addressing some limitations of the two-stage approach. These methodologies leverage advanced GNNs to aggregate node representations within each graph sample and utilize pooling techniques to generate graph representations. Moreover, some techniques craft scoring functions based on the original definition of anomalies, which refers to instances diverging from a large number of others, to enhance the accuracy of evaluating graph sample abnormalities. Furthermore, influenced by unsupervised or semi-supervised graph representation learning, their training processes also reduce dependency on label information to some extent.

Despite their significant success, we believe there are still the following challenges that urgently need to be addressed: ***Challenge 1:*** *Inadequate extraction of complex anomalous information within graphs,* stemming from the limited receptive field of GNNs. Fig. 1(a) illustrates the presence of potential attribute or structural anomalies denoted by the red nodes and red dashed areas in the graph. Unfortunately, due to the restricted receptive field of GNNs, when attempting to propagate this crucial anomalous information to aggregate a reliable representation of the target node, it is often overlooked as it is located too far from the target node. Consequently, the resulting node representation is of low quality, which adversely affects the overall quality of the graph representation, as the latter is derived from the pooled node representations. ***Challenge 2:*** *Insufficient exploration of explicit modeling of inter-graph information,* which is related to the general paradigm of existing GLAD approaches. Existing GLAD methods generate graph-level representations in the embedding space by utilizing node representations within each graph. Subsequently, these methods attempt to determine appropriate decision boundaries within that space. However, these methods treat each graph sample as an independent entity in the embedding space, ignoring the potential relationships between graphs. As shown in Fig. 1(b), the relationships between graphs, such as similarity or dissimilarity, can provide additional insights for anomaly detection tasks. Therefore, we emphasize the critical importance of explicitly modeling cross-graph interaction information from the perspective of the graphs themselves. ***Challenge 3:*** *Limited and incomplete development of anomaly scoring mechanisms.* Certain methods, particularly those two-stage GLAD models that integrate traditional machine learning techniques as detectors, tend to oversimplify the anomaly detection task by converting it into a binary graph classification problem. This simplistic approach fails to fully consider the diverse nature and characteristics of anomalies throughout the detection process. Consequently, there is a need for a more holistic and sophisticated construction of anomaly scoring mechanisms to ensure the precise identification and classification of various anomaly types within graph data.

To alleviate the previously mentioned challenges, in this paper, we propose a novel model called Dual-view **G**raph-**o**f-**G**raph **R**epresentation **L**earning with Graph Transformer (GoGRL) for GLAD tasks. Our goal is to thoroughly explore the information contained in graph samples from both intra-graph and inter-graph perspectives, in order to obtain high-quality and discriminative graph representations for graph-level anomaly detection, without relying on true anomaly labels. **To address Challenge 1 and Challenge 3,** we focus on two crucial aspects: comprehensive exploration of intra-graph information and accurate measurement of anomalies within individual graphs. Specifically, to fully extract the intricate anomalous information concealed within graphs, we propose a graph embedding approach augmented with graph transformers. This approach improves upon traditional GNNs by incorporating graph transformers, which expand the receptive field of standard GNNs while preserving the node embeddings generated by GNNs. Furthermore, building on the foundational assumption that anomalous nodes within a single graph are challenging to reconstruct, we employ an intra-graph reconstruction loss to refine the model and produce plausible anomaly scores. **To tackle Challenge 2 and Challenge 3,** our approach is to explicitly model inter-graph information interactions and capture the abnormal signals present between graphs. Specifically, we introduce a dual-view graph-of-graph representation learning module. This module generates two distinct Graph-of-Graph views, each grounded in the structural and attribute information of the graph data. By doing so, it thoroughly models the relationships between graphs, thereby effectively trapping anomalous information from a holistic viewpoint. In parallel with the intra-graph reconstruction loss, an inter-graph reconstruction loss is employed to pinpoint potential anomalies that span across multiple graphs, which helps identify potential anomalies that span across multiple graphs, further strengthening the detection capabilities of our model.

To summarize, the main contributions of our work are as follows:

- We present a Transformer-enhanced graph encoder that integrates the Transformer architecture with structural positional encoding to augment the representational power of traditional GNNs, resulting in more discriminative intra-graph representations. This approach not only extends the receptive field of GNNs but also enhances the model's sensitivity to structural anomalies, thereby facilitating a more comprehensive exploration of intra-graph information.
- We design a dual-view graph-of-graph representation learning module, which represents our initial attempt to explicitly model inter-graph information. This endeavor involves utilizing both structural and attribute information from graphs to comprehensively construct cross-graph relationships and uncover anomalous signals within them.

- We conduct comprehensive experiments on real-world datasets to assess the effectiveness of our proposed model. The results showcase the greater performance of GoGRL and confirm the validity of our approach in addressing the challenges.

The rest of this paper is organized as follows: Section 2 offers an in-depth review of related works relevant to our research problem and the current technologies. Section 3 outlines our research problem and provides detailed information about the overall pipeline and components of our framework. In Section 4, we discuss the experimental results from our study. Lastly, Section 5 presents our conclusions and summarizes our findings.

2. Related work

2.1. Graph transformer

The Transformer was initially introduced in the field of machine translation, with its pivotal concept being the attention mechanism. Recently, numerous endeavors have been made to extend the Transformer into the domain of graph representation learning because its global attention mechanism effectively alleviates the receptive field issue in graph neural networks, thus enhancing the quality of representation learning. Graph Transformer (Dwivedi & Bresson, 2021) presented an early model demonstrating how to extend the Transformer architecture to graphs. They utilized Laplacian eigenvectors as an absolute encoding and computed attention within the immediate neighborhood of each node. Similarly, SAN (Kreuzer et al., 2021) also employed Laplacian eigenvectors for absolute encoding but performed attention calculation across the entire graph while distinguishing between true and created edges. Many other graph Transformer methods, such as the one proposed by Shaw et al. (2018), incorporate both absolute and relative encoding. This approach integrates representations of relative positions or distances between nodes directly into the self-attention mechanism, in contrast to absolute encoding, which is applied only once to input node features. Mialon et al. (2021) introduced a relative encoding using graph kernels to influence self-attention calculations, enabling the incorporation of positional information into Transformers through the choice of kernel function. GraphGPS (Rampášek et al., 2022) systematically examined the impact of various positional encodings and proposed an innovative architecture that decoupled local real-edge aggregation from the fully-connected Transformer mechanism, thereby enhancing the scalability of the model. NodeFormer (Wu et al., 2022) introduced a novel all-pair message passing scheme, leveraging a kernelized Gumbel-Softmax operator to significantly reduce the time complexity of representation learning, enabling the efficient extension of the learning process to large-scale graphs.

To sum up, in this study, we merge the advantages of message passing Graph Neural Networks (GNNs) and the Transformer architecture. By leveraging Transformer to expand the receptive field of GNNs and utilizing GNNs to complement Transformer with structural information, we aim to obtain more discriminative graph-level representations for GLAD.

2.2. Graph-level anomaly detection

Graph-level anomaly detection methods aim to differentiate between normal graphs and anomalous graphs within a given set of graph data. Existing research can be broadly categorized into two types: two-stage methods and end-to-end methods. Early two-stage methods typically use graph kernels, such as Weisfeiler–Leman kernel (WL) (Shervashidze et al., 2011), propagation kernel (PK) (Neumann et al., 2016), to learn graph representations, which are then utilized by pre-existing anomaly detectors, such as iF (Liu et al., 2008), one-class SVM (Manevitz & Yousef, 2001), LOF (Breunig et al., 2000), to identify anomalous graphs. Recently, more methods have begun to explore integrating graph representation learning and anomaly severity assessment into a unified framework to achieve end-to-end graph-level anomaly detection. They adopt powerful graph neural networks to learn graph embeddings and, based on the extracted representations, identify anomalous samples within the given graph data. GLocalKD (Ma et al., 2022) adopts GNNs as encoders and implements random knowledge distillation. GOOD-D (Liu et al., 2023) designs a novel non-perturbative graph data augmentation method and uses GIN as an encoder for graph-level anomaly detection. CVTGAD (Li et al., 2023a) introduces a cross-attention mechanism to simultaneously capture intra-graph and inter-graph correlations. HimNet (Niu et al., 2023) introduces a memory network to separately optimize node-level and graph-level representations from a global perspective, resulting in more discriminative graph-level representations.

To sum up, current methods for addressing GLAD still face limitations, such as the finite receptive field issue from GNNs' neighborhood aggregation mechanism and the vague modeling of inter-graph relationships. To bridge these gaps, our method not only effectively expands the GNNs' receptive field but also explicitly captures inter-graph relationships, providing a comprehensive solution.

3. Methodology

In this section, we introduce our proposed Dual-view Graph-of-Graph Representation Learning with Graph Transformer (GoGRL) method. The overall pipeline of GoGRL is illustrated in Fig. 2. For each input graph sample, we first generate node embeddings using an enhanced Transformer graph encoder, and then generate initial graph embeddings through a readout function. After that, we construct two different views: the feature graph-of-graph view and the structure graph-of-graph view, each of which captures the attribute and structural correlations between input graph samples, respectively. Simultaneously, we attach the initial graph embeddings to the nodes in the feature/structure Graph-of-Graph (GoG) and refine them using two different GNNs-based encoders. Then, we perform reconstruction estimation at two levels: intra-graph reconstruction and GoG reconstruction, and perform approximation learning between the two GoG views. Ultimately, by recalling the reconstruction errors at both levels, we calculate an aggregated score that serves as a measure for evaluating the anomaly detection result.

In the following subsections, we will first introduce some necessary notations and the problem definition of the GLAD task. Following that, we will provide a detailed description of the design of our proposed GoGRL. Furthermore, to provide a clearer illustration of our method, we summarize the overall procedure of the GoGRL framework at the end of this section.

3.1. Notations and problem definition

We start by presenting the notations utilized throughout this paper. In this paper, matrices are represented by bold uppercase letters (e.g., $\mathbf{X}$), vectors by bold lowercase letters (e.g., $\mathbf{x}$), and sets by calligraphic fonts (e.g., $\mathcal{V}$). For convenience, the main notations used in this paper are summarized in Table 1.

Attributed Network. Given an attributed network $G = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}(|\mathcal{V}| = n)$, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}(|\mathcal{E}| = m)$, and $\mathbf{X} \in \mathbb{R}^{n \times d}$ denote the node set, edge set, and attribute matrix, respectively. The ith row vector $\mathbf{x}_i \in \mathbb{R}^d$ of the attribute matrix indicates the attribute information of the ith node. A binary adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ represents the network's structural information, where $a_{i,j} = 1$ if $e_{i,j} \in \mathcal{E}$ and $a_{i,j} = 0$ otherwise.

Unsupervised Graph-level Anomaly Detection. This paper focuses on the Graph-level Anomaly Detection (GLAD) problem. Specifically, given a collection of k graphs $\mathcal{G} = \{G_1, G_2 ..., G_k\}$ containing both normal and anomalous graphs, our objective is to learn an anomaly estimation function $f(\cdot)$ that generates an anomaly score $score(G_i) = f(G_i)$ for each $G_i \in \mathcal{G}$. In practice, an anomalous graph, which deviates significantly from the majority of graphs in the set, will receive a higher score from our scoring function.

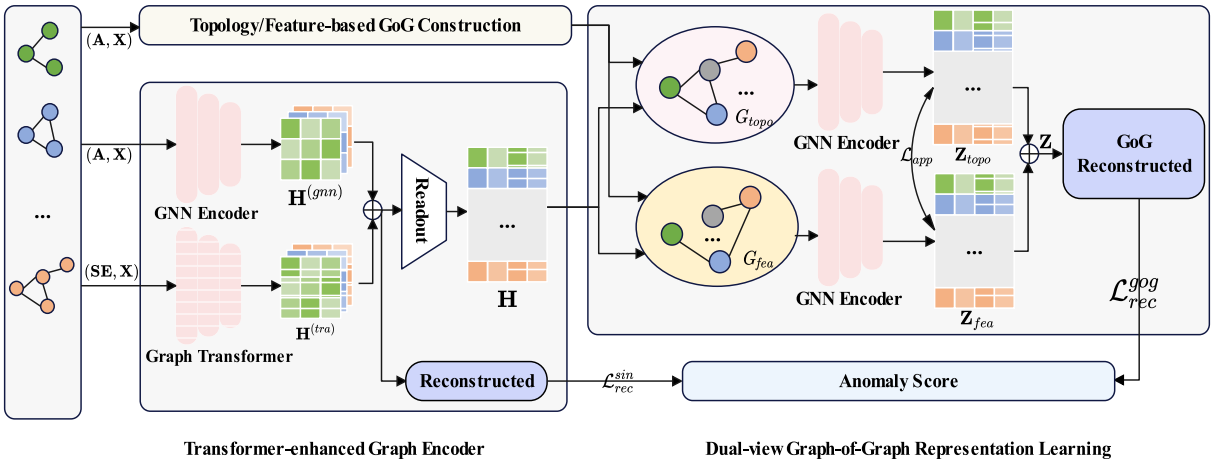


Fig. 2. The overall of proposed GoGRL. In the Transformer-enhanced graph encoder module, for each input graph sample, we first generate and fuse two representations for all nodes in the graph. We then use a readout function to obtain a graph-level representation for each graph, which is used as the initial graph representation for graph propagation in the graph-of-graph process. In *dual-view graph-of-graph representation learning module*, we propagate the initial graph representations separately in the constructed graph-of-graph perspectives to obtain the final graph representations optimized through the graph-of-graph process. *The anomaly scoring module* calculates anomaly scores for each graph sample based on the representations obtained from the two modules. Specifically, it conducts intra-graph reconstruction and graph-of-graph reconstruction, and computes the anomaly score for each graph sample based on the reconstruction errors.

Table 1
Key notations and explanations of GoGRL framework.

Notations	Explanation
$\mathcal{G} = \{G_1, G_2 ..., G_k\}$	The input graph collection with k graph samples.
$G_{topo} = (\mathbf{H}, \mathbf{A}_{topo})$	The topology-based graph-of-graph (GoG) view.
$G_{fea} = (\mathbf{H}, \mathbf{A}_{fea})$	The feature-based graph-of-graph (GoG) view.
$\mathbf{H}$, $\mathbf{h}_{G_i}$	The transformer-enhanced graph representation matrix, the transformer-enhanced graph embedding of G_i.
$\mathbf{Z}_{topo}$, $\mathbf{Z}_{fea}$, $\mathbf{Z}$	The graph representation matrix of G_{topo}, the graph representation matrix of G_{fea}, the final graph representation matrix.

3.2. Transformer-enhanced graph embedding

In this subsection, our aim is to design the Transformer-enhanced graph encoder with the purpose of effectively revealing potential anomaly patterns within individual graphs. This design enables the generation of initial graph embeddings specifically for graph-of-graph message propagation. The prevalent choice in previous GLAD researches has been to use GNNs-based encoders and readout functions to acquire graph embeddings, owing to their natural process. However, the quality of these generated graph embeddings is constrained by the small receptive field caused by the neighborhood aggregation mechanism. While some studies (Li et al., 2023a) have tackled this issue by enhancing the receptive field of GNNs using Transformers, they solely rely on the feature information of graph nodes for attention aggregation. This approach might lead to transformer-based graph embeddings missing anomalous structural patterns present within the graph. Hence, in our design, we employ GNNs encoders and introduce structural encoding in the graph Transformer to produce GNN-based embeddings and Transformer-based embeddings for each node in the input graph samples, respectively. These representations are then combined and obtained as initial graph embeddings through a readout function.

GNNs-based Embeddings. In this paper, to calculate the preliminary node embeddings, we introduce a GNN encoder. Specifically, due to its powerful information extraction capability, the Graph Isomorphism Network (GIN) (Xu et al., 2019) is adopted as the GNNs encoder in this work. It is worth noting that our framework is flexible and can accommodate the use of other Graph GNN encoders as well. The propagation rule of GIN in the lth layer can be expressed as:

$$\mathbf{h}_i^{(gnn,l)} = \text{MLP}(\mathbf{h}_i^{(gnn,l-1)} + \sum_{v_j \in \mathcal{N}(v_i)} \mathbf{h}_j^{(gnn,l-1)}), \tag{1}$$

where $\mathbf{h}_i^{(gnn,l)}$ is the embedding of node v_i at the lth layer of GNN encoder, $\mathcal{N}(v_i)$ is the set of first-order neighborhood nodes of node v_i, and MLP is a multi-layer perceptron network. In our model, we set $\mathbf{h}_i^{(gnn,0)} = \mathbf{x}_i$ and take the output of Lth layer $\mathbf{h}_i^{(gnn,L)}$ as the final GNNs-based embeddings, which we denote as $\mathbf{h}_i^{(gnn)}$.

Transformer-based Embeddings. Here, we introduce the process of generating transformer-based embeddings using a Transformer that incorporates positional encoding. The design aims to integrate the graph structural information that is typically overlooked by traditional Transformers into the calculation of attention scores. Specifically, as defined by Dwivedi et al. (2022), we utilize a d_{se}-step random walk-based graph diffusion for the d_{se} dimensional structural encoding $\mathbf{se}_i$ of node v_i:

$$\mathbf{se}_i = [\mathbf{RW}_{ii}, (\mathbf{RW}^2)_{ii}, \dots (\mathbf{RW}^{d_{se}})_{ii}] \in \mathbb{R}^{d_{se}}, \tag{2}$$

where $\mathbf{RW} = \mathbf{AD}^{-1}$ is the random walk transition matrix. To simplify the computation, we use only the probability of node i returning to itself at each step of the random walk to construct the structural encoding.

After obtaining the positional encodings $\mathbf{SE}$ for each node in the graph, we concatenate them with the initial attributes of these nodes and feed them into the projection network to obtain three parametric matrices: query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$, and value matrix $\mathbf{V}$, which are required by the Transformer. Using these three matrices, we calculate attention scores for all nodes in the target graph through an all-pair attention mechanism, and generate Transformer-based embeddings based on these scores. Specifically, the attention aggregation process we adopt is defined as follows:

$$\mathbf{Q} = \text{MLP}_Q([\mathbf{X} \parallel \mathbf{SE}]),\ \mathbf{K} = \text{MLP}_K([\mathbf{X} \parallel \mathbf{SE}]),\ \mathbf{V} = \text{MLP}_V([\mathbf{X} \parallel \mathbf{SE}]),$$
$$\mathbf{H}^{(tra)} = softmax(\frac{\mathbf{QK}^\top}{\sqrt{d_k}})\mathbf{V}, \tag{3}$$

where $\text{MLP}_Q, \text{MLP}_K, \text{MLP}_V$ represent MLPs with different parameters, used to map input embeddings into the same projection space. $[\cdot \parallel \cdot]$

represents the concatenate operation and d_k is a scaling factor set to the dimension of $[\mathbf{X} \parallel \mathbf{SE}]$. The ith row $\mathbf{h}_i^{(tra)}$ of $\mathbf{H}^{(tra)}$ is the final transformer-based embedding of the node v_i.

Graph Embeddings Readout. Once we have obtained the two types of node embeddings, we require a readout function to generate initial graph embeddings. In this work, we adopt global average pooling as the readout function, which can be represented as:

$$\mathbf{h}_{G_i} = \frac{1}{|\mathcal{V}_{G_i}|} \sum_{v_j \in \mathcal{V}_{G_i}} [\mathbf{h}_j^{(gnn)} \parallel \mathbf{h}_j^{(tra)}], \tag{4}$$

where $\mathcal{V}_{G_i}$ is the node set of input graph G_i, and $[\cdot \parallel \cdot]$ represents the concatenate operation. Once we obtain all the graph embeddings $\mathbf{h}_{G_i}$, we aim to combine them into $\mathbf{H}$, which represents the initial graph embeddings for the subsequent graph-of-graphs construction process.

Design Discussion. In this subsection, we propose a Transformer-enhanced graph encoder with the primary objective of fully leveraging the latent information within the graph. Specifically, our method represents an initial attempt to integrate Graph Transformers into the GLAD task by combining the standard Transformer architecture with random walk-based positional encodings. This design offers the following unique advantages:

(1) Enhanced Receptive Field: Traditional graph-level anomaly detection methods based on GNNs often suffer from limited receptive fields. To address this issue, we introduce the Transformer to enhance the receptive field of GNNs. By combining the graph embeddings encoded by GNNs with those encoded by the Transformer, we greatly improve the receptive field of GNNs, enabling them to capture more comprehensive contextual information.

(2) Structural Encoding: In contrast to the standard Transformer architecture, which primarily relies on the attribute information of nodes in the graph for self-attention computation, the Transformer we utilize consciously incorporates structural encoding into the self-attention computation process. This integration supplements additional structural information, allowing for a more holistic understanding of the graph's underlying relationships.

By incorporating these design elements, our proposed Transformer-enhanced graph encoder overcomes the limitations of traditional GNNs-based methods and provides a more comprehensive and effective approach for graph-level anomaly detection. In future work, we plan to further explore the use of Graph Transformers in more complex anomaly detection scenarios, including scaling to larger graphs and refining the standard self-attention mechanism.

3.3. Dual-view graph-of-graph construction and propagation

As mentioned previously, existing research has not fully explored the potential relationships between graphs, primarily due to the lack of explicit modeling of complex relationships between graphs. To tackle this issue, we propose a dual-view graph-of-graph representation learning network. This module is mainly divided into two parts: graph-of-graph construction and graph-of-graph propagation. The two distinct graph-of-graph views we construct, based on topology and features, reflect complementary and consistent relationships between graphs from the structural and attribute perspectives. At the same time, message passing is performed based on the two constructed graph-of-graph views to obtain discriminative graph representations. For convenience, we will refer to Graph-of-Graph as GoG in the following sections.

Graph of Graphs Construction. In this subsection, our goal is to construct two high-level graphs to fully explore the connections between graphs from different perspectives. Specifically, given a set of graphs $\mathcal{G}$, where each graph $G \in \mathcal{G}$ is represented by a node in the high-level graph, and if two graphs are similar, they are connected by an edge in the high-level graph. In our work, we measure the similarity between two graphs based on both their topological similarity and feature similarity, as we believe that graphs with similar topology or similar attributes often belong to the same class, such as two graphs with dense connections both likely being anomalous samples. Here, we utilize two different graph kernel functions to quantify the similarity between graph pairs construct GoGs and leverage these measurements to construct GoGs.

Formally, let $\mathbf{S}^{top}$ and $\mathbf{S}^{fea}$ denote the matrices for topological and feature similarity, respectively. Each entry $\mathbf{s}_{ij}^{top}$ and $\mathbf{s}_{ij}^{fea}$ captures the topological and feature similarity between graphs (G_i, G_j), computed using the kernel functions ϕ and ψ as follows, respectively:

$$\mathbf{s}_{ij}^{top} = \phi(G_i, G_j),\ \mathbf{s}_{ij}^{fea} = \psi(G_i, G_j), \tag{5}$$

where (G_i, G_j) denotes a pair of graph samples from the dataset for which similarity is to be measured. The functions ϕ and ψ represent the kernels employed to compute topological similarity and feature similarity, respectively.

More specifically, the shortest path kernel function (Borgwardt & Kriegel, 2005) is chosen as ϕ for the computation of topological similarity. Initially, the graphs G_i and G_j are transformed into their corresponding shortest-path graphs $SP_i = (\mathcal{V}_i, \tilde{\mathcal{E}}_i)$ and $SP_j = (\mathcal{V}_j, \tilde{\mathcal{E}}_j)$ using the Floyd-transformation. The shortest-path graph SP_i retains the same node set as the input graph G_i, while the set $\tilde{\mathcal{E}}_i$ contains the shortest path lengths for all node pairs in the graph G_i, and the same applies to SP_j. Subsequently, based on these shortest-path graphs, we employ the shortest-path kernel $k_{shortest\ path}(SP_i, SP_j)$ to compute the topological similarity between G_i and G_j. More specifically, following the approach described in literature (Borgwardt & Kriegel, 2005), we select $k_{walk}(\cdot,\cdot)$, which we refer to as shortest path index kernel, as a specific implementation of the shortest-path kernel. This kernel is used to iterate over the elements in $\tilde{\mathcal{E}}_i$ and $\tilde{\mathcal{E}}_j$, ultimately obtaining the topological similarity between the two graphs. The formula for topological similarity as given in Eq. (5) can be reformulated as indicated below:

$$\mathbf{s}_{ij}^{top} = \phi(G_i, G_j) = k_{shortest\ path}(SP_i, SP_j) = \sum_{e_m \in \tilde{\mathcal{E}}_i} \sum_{e_n \in \tilde{\mathcal{E}}_j} k_{walk}(e_m, e_n), \tag{6}$$

where e_m and e_n are elements of sets $\tilde{\mathcal{E}}_i$ and $\tilde{\mathcal{E}}_i$, respectively, representing the shortest path lengths between node pairs in their corresponding graphs. The explicit form of $k_{walk}(e_m, e_n)$ is as follows:

$$k_{walk}(e_m, e_n) = \begin{cases} 1, & \text{if } e_m = e_n \\ 0, & \text{if } e_m \neq e_n \end{cases}. \tag{7}$$

Subsequently, cosine similarity is utilized for the computation of feature similarity. The initial graph embeddings $\bar{\mathbf{x}}_i$ and $\bar{\mathbf{x}}_j$ for graphs G_i and G_j are derived by applying average pooling to their respective node attributes. As a result, the feature similarity term in Eq. (5) is reformulated as:

$$\mathbf{s}_{ij}^{fea} = \psi(G_i, G_j) = cosine\ similarity(\bar{\mathbf{x}}_i, \bar{\mathbf{x}}_j) = \frac{\bar{\mathbf{x}}_i \bar{\mathbf{x}}_j}{\| \bar{\mathbf{x}}_i \|_2 \| \bar{\mathbf{x}}_j \|_2}, \tag{8}$$

where $\| \cdot \|_2$ represents L2 norm of the vector.

Then we construct two kNN graphs G_{topo} and G_{fea} by connecting each graph with its top-k similar graphs based on the similarity matrix $\mathbf{S}^{top}$ and $\mathbf{S}^{fea}$, respectively. At this point, the structural information of the topology-based GoG and the feature-based GoG, represented as $\mathbf{A}_{topo}$ and $\mathbf{A}_{fea}$, has been constructed. The description of the attribute information and message passing within these two graphs will be provided in the next section.

Graph of Graphs Propagation. In this subsection, we aim to elucidate the process of propagating the graph embeddings $\mathbf{H}$ obtained through Eq. (4) throughout the GoG and refining the graph representations via an approximation loss function. The motivation behind this design is that normal graph samples are more likely to aggregate graph representations of the same class when transmitting messages across the constructed GoG view, thereby making the updated graph embeddings closely approximate the original graph embeddings. Conversely, anomalous graph instances may diverge in their updated graph embeddings from the initial embeddings, attributable to the amalgamation of embeddings from normal graphs. These disparate manifestations

between these two categories of graphs furnish compelling evidence for graph-level anomaly detection.

Formally, after attaching the graph embeddings $\mathbf{H}$ onto the two GoG views constructed in the previous section, we obtain two distinct GoGs, denoted as $G_{topo} = (\mathbf{H}, \mathbf{A}_{topo})$ and $G_{fea} = (\mathbf{H}, \mathbf{A}_{fea})$. Subsequently, for these two different graph perspectives, we employ two GNNs with non-shared parameters. Here, GIN continues to serve as the backbone network for learning node representations within the GoG views. The lth layer GoG propagation can be formalized as follows:

$$\mathbf{z}_i^{(l)} = \mathrm{MLP}(\mathbf{z}_i^{(l-1)} + \sum_{v_j \in \mathcal{N}(v_i)} \mathbf{z}_j^{(l-1)}), \tag{9}$$

where $\mathbf{z}_i^{(l)}$ and $\mathbf{z}_i^{(l-1)}$ respectively represent the lth and $(l-1)$-th layer representations of the ith node within the GoG, which in turn reflect the graph representation of G_i. $\mathcal{N}(v_i)$ is the set of first-order neighborhood nodes of node v_i, and MLP is a multi-layer perceptron network. The graph embeddings of the entire graph can be denoted as $\mathbf{Z}$. It is worth noting that the propagation process in Eq. (9) do not differentiate between graph perspectives since they share similar procedures. The main distinction between these two propagation processes lies in the different structural information they incorporate.

After obtaining the graph representations $\mathbf{Z}_{topo}$ and $\mathbf{Z}_{fea}$ for the two perspectives according to Eq. (9), we aim to calculate and minimize the approximation error between these two representations. The motivation behind this design stems from the assumption that normal graphs should be connected to similar graphs in both perspectives, thus leading to consistent representations. However, anomalous graph samples may exhibit inconsistencies in structure or attributes, which could result in connections to different neighbors across the two perspectives, causing discrepancies in their representations. Therefore, minimizing the difference between these two representations can help extract more discriminative features for the subsequent reconstruction process.

Formally, the approximation error is calculated through the following $\mathcal{L}_{app}$ loss:

$$\mathcal{L}_{app} = \|\mathbf{Z}_{topo} - \mathbf{Z}_{fea}\|_F^2, \tag{10}$$

where $\|\cdot\|_F$ denotes Frobenius-norm.

Design Discussion. The Dual-view Graph-of-Graph Representation Learning module presented in this section significantly advances the understanding of inter-graph relationships. The module offers the following distinct benefits:

(1) Extensive Relationship Examination: Our module differs from approaches that do not consider inter-graph connections by simultaneously analyzing anomalous features within individual graphs and integrating inter-graph information within the GLAD framework. This approach resonates with the fundamental concept of graph-level anomaly detection, which posits that anomalies can originate from both inherent graph attributes and interactions with other graphs.

(2) Clear and Extensive Modeling Strategy: Our method stands out against those that model inter-graph relationships by providing a lucid and expansive modeling strategy. We explicitly delineate the relationships among graph instances, encompassing both attribute and structural dimensions. This perspective uncovers nuanced insights into the detection of potential attribute and structural anomalies intertwined within the dataset.

These design features contribute to our module's efficacy in robustly and extensively navigating graph relationships and identifying graph-level anomalies.

3.4. Anomaly scoring module

Corresponding to the challenges we discussed, the calculation of anomaly scores should encompass two distinct perspectives: intra-graph anomaly scores and inter-graph anomaly scores. In earlier sections, we have highlighted the graph representation learning process from both intra-graph and inter-graph perspectives to obtain more discriminative graph representations. A natural progression from this is how to effectively apply these high-quality graph representations to generate anomaly scores for graphs. In previous works on node-level anomaly detection, reconstruction loss has been considered an effective measure. Taking inspiration from this, for intra-graph and inter-graph anomaly scores, we respectively utilize single-graph reconstruction loss and graph-of-graph reconstruction loss for computation. In the following, we will detail the motivations and computational methods for these two loss functions, as well as how they are employed to construct our anomaly scores.

Single Graph Reconstruction Loss. The motivation for designing this loss arises from our conceptualization of intra-graph anomalies. We posit that, from the perspective of an individual graph, if a substantial number of anomalous nodes – characterized by higher reconstruction errors – are present, the overall reconstruction loss of the graph will increase, thereby indicating the graph as anomalous. Minimizing this loss during training encourages the model to capture the patterns of normal graphs and preserve their semantic integrity. As a result, during inference, the model is expected to produce higher reconstruction errors for anomalous graphs, which deviate from the normal patterns encountered during training.

Specifically, for a given graph G_i, in Section 4.1, we already have two types of intra-graph node representations: GNNs-based embeddings and Transformer-based embeddings, denoted as $\mathbf{H}_i^{(gnn)}$ and $\mathbf{H}_i^{(tra)}$, respectively. After merging them, we perform separate structural reconstruction and attribute reconstruction. By combining the reconstruction errors of nodes contained within this graph, we can generate intra-graph anomaly scores from the perspective of a single graph. Specifically, we feed the merged intra-graph node representations $[\mathbf{H}_i^{(gnn)} \| \mathbf{H}_i^{(tra)}]$ into the structural decoder $Dec_{single}^{str}(\cdot)$ and the attribute decoder $Dec_{single}^{attr}(\cdot)$. In this manner, the single graph reconstruction loss can be reformulated as:

$$\mathcal{L}_{rec}^{sin} = \| \mathbf{A}_i - Dec_{single}^{str}([\mathbf{H}_i^{(gnn)} \| \mathbf{H}_i^{(tra)}]) \|_F^2 + \| \mathbf{X}_i - Dec_{single}^{attr}([\mathbf{H}_i^{(gnn)} \| \mathbf{H}_i^{(tra)}]) \|_F^2, \tag{11}$$

where $[\cdot \| \cdot]$ denotes the concatenation operation, while $\|\cdot\|_F$ represents the Frobenius norm.

Graph-of-Graph Reconstruction Loss. Inspired by relevant work on anomalous node detection (Jin et al., 2024), we conceptualize each graph sample as a node in the higher-level graph of graphs. Consequently, it is natural for us to consider performing attribute and structural reconstruction on the GoGs, which characterizes the relationships between different graphs, and to use the reconstruction errors to indicate inter-graph anomaly scores. Similar to the single-graph reconstruction loss, this loss guides our model to capture the patterns of a significant number of normal samples and assigns higher reconstruction errors to anomalous nodes in the GoGs, corresponding to anomalous graph samples, during inference.

Specifically, for graph-of-graph reconstruction, we require three decoders, denoted as $Dec_{topo}^{str}(\cdot)$, $Dec_{fea}^{str}(\cdot)$, and $Dec_{gog}^{attr}(\cdot)$ to decode the graph-of-graph embeddings $\mathbf{Z}$ that integrate different information from two graph-of-graph views, reconstructing the structures of G_{topo} and G_{fea} as well as the attribute information. The graph-of-graph reconstruction loss can be formalized as follows:

$$\mathcal{L}_{rec}^{gog} = \|\mathbf{A}_{topo} - Dec_{topo}^{str}(\mathbf{Z})\|_F^2 + \|\mathbf{A}_{fea} - Dec_{fea}^{str}(\mathbf{Z})\|_F^2 + \|\mathbf{H} - Dec_{gog}^{attr}(\mathbf{Z})\|_F^2, \tag{12}$$

where $\mathbf{Z} = (\mathbf{Z}_{topo} + \mathbf{Z}_{fea})/2$, while $\|\cdot\|_F$ represents the Frobenius norm.

Anomaly Scoring and Detection. After introducing the two loss functions used for model optimization, we now detail the process of generating anomaly scores for each graph sample and explain how these scores are utilized to determine whether a graph sample is anomalous.

First, by applying Eqs. (11) and (12), we generate the intra-graph anomaly score and the inter-graph anomaly score, respectively. Specifically, on one hand, Eq. (11) provides the reconstruction errors for all nodes within a single graph. By summing these node-level errors, we

Algorithm 1: The overall procedure of GoGRL

Input: A set of attributed network $\mathcal{G} = \{G_1, G_2 ..., G_k\}$, the kernel function ϕ and ψ, number of iterations T.
Output: Anomaly score ranking list.
// Training phase
Construct two adjacency matrices $\mathbf{A}_{topo}$ and $\mathbf{A}_{fea}$ for graph-of-graph structures via ϕ and ψ;
for *Iteration 1,2,...,T* **do**
  Calculate GNNs-based embeddings $\mathbf{H}_i^{(gnn)}$ of G_i via Eq. (1);
  Calculate Transformer-based embeddings $\mathbf{H}_i^{(tra)}$ of G_i via Eqs. (2)–(3);
  Acquire graph embeddings $\mathbf{H}$ via Eq. (4);
  Construct two different GoGs $G_{topo} = (\mathbf{H}, \mathbf{A}_{topo})$ and $G_{fea} = (\mathbf{H}, \mathbf{A}_{fea})$;
  Calculate graph representations $\mathbf{Z}_{topo}$ and $\mathbf{Z}_{fea}$ after GoG propagation via Eq. (8);
  Acquire graph embeddings $\mathbf{Z} = (\mathbf{Z}_{topo} + \mathbf{Z}_{fea})/2$;
  Calculate $\mathcal{L}_{app}$, $\mathcal{L}_{rec}^{sin}$ and $\mathcal{L}_{rec}^{gog}$ via Eqs. (9)–(11);
  Back-propagate $\mathcal{L}_{app}$, $\mathcal{L}_{rec}^{sin}$ and $\mathcal{L}_{rec}^{gog}$ to update parameter;
end
// Inference phase.
Calculate GNNs-based embeddings $\mathbf{H}_i^{(gnn)}$ of G_i via Eq. (1);
Calculate Transformer-based embeddings $\mathbf{H}_i^{(tra)}$ of G_i via Eqs. (2)–(3);
Acquire graph embeddings $\mathbf{H}$ via Eq. (4);
Construct two different GoGs $G_{topo} = (\mathbf{H}, \mathbf{A}_{topo})$ and $G_{fea} = (\mathbf{H}, \mathbf{A}_{fea})$;
Calculate graph representations $\mathbf{Z}_{topo}$ and $\mathbf{Z}_{fea}$ after GoG propagation via Eq. (8);
Acquire graph embeddings $\mathbf{Z} = (\mathbf{Z}_{topo} + \mathbf{Z}_{fea})/2$;
Calculate intra-graph anomaly score $\text{score}_{intra}(G_i)$ for G_i via Eq. (10);
Calculate inter-graph anomaly score $\text{score}_{inter}(G_i)$ for G_i via Eq. (11);
Calculate final anomaly score $\text{score}(G_i)$ for G_i via Eq. (12);
return *Anomaly score ranking list.*

obtain the intra-graph anomaly score $\text{score}_{intra}(G_i)$ for graph G_i. On the other hand, after applying Eq. (11), each term in the loss function corresponds to the reconstruction loss of each graph on the two GoGs, which we interpret as the inter-graph anomaly score $\text{score}_{inter}(G_i)$ for graph G_i.

Then, with both intra-graph and inter-graph anomaly scores, the overall anomaly score of a graph is calculated as follows:

$$\text{score}(G_i) = \lambda \cdot \text{score}_{intra}(G_i) + (1 - \lambda) \cdot \text{score}_{inter}(G_i), \tag{13}$$

where λ controls the balance between the two types of anomaly scores.

Ultimately, after computing the anomaly scores, in contrast to the methods described in Challenge 3, which apply traditional binary classifiers to simplify anomaly detection as a binary graph classification task, we adopt a threshold-based classification approach leveraging the anomaly scores generated by Eq. (12). Specifically, once the anomaly scores for each graph sample are determined, we first rank all samples in descending order based on their scores, and subsequently classify the top N samples with the highest anomaly scores as anomalies. Compared to traditional binary classifiers, our approach offers several key advantages: (1) Rather than relying on model-derived probabilities, we classify based on computed anomaly scores, which provides a more comprehensive assessment as the scores are derived by fully considering both intra-graph and inter-graph perspectives. (2) The threshold N used in our classification method is highly flexible, allowing for adjustments to suit complex real-world scenarios. For instance, in fraud detection, a higher threshold can be set to capture a greater number of suspicious samples, thereby minimizing the risk of overlooking anomalous cases.

3.5. Overall procedure and time complexity analysis

In this subsection, we introduce the overall procedure of the proposed GoGRL algorithm and analyze the time complexity of its key processes. Overall, the proposed GoGRL model is divided into two phases: the training phase and the inference phase. Algorithm 1presents the overall procedure of the GoGRL framework.

In the training phase, we first construct two adjacency matrices for the GoG structures using kernel functions ϕ and ψ. Specifically, ϕ represents the topology similarity function based on the shortest-path kernel, with a computational complexity of $\mathcal{O}(n^3)$ (Borgwardt & Kriegel, 2005), while ψ computes feature cosine similarity with a complexity of $\mathcal{O}(d)$. Given that pairwise calculations are required across all graphs, the time complexity for constructing these adjacency matrices is $\mathcal{O}(k^2(n^3 + d))$, where k represents the number of graphs, n is the maximum number of nodes across these graphs, and d is the dimensionality of the initial node embeddings. Although the computational cost seems substantial, this process is a one-time effort, as the computed similarity matrices can be stored for future use. This significantly mitigates the computational burden, reducing the challenges associated with repeated calculations.

After constructing the two graph-of-graph structures, we iteratively perform several steps. First, we calculate the GNN-based embedding $\mathbf{H}^{(gnn)}$ for each graph using the GIN model, which has a time complexity of $\mathcal{O}(k \cdot |\mathcal{V}_i| \cdot |\mathcal{E}_i|)$, where k is the number of graphs, $|\mathcal{V}_i|$ is the number of nodes, and $|\mathcal{E}_i|$ is the number of edges. Then, we compute the Transformer-based embedding $\mathbf{H}^{(tra)}$, with a time complexity of $\mathcal{O}(k \cdot |\mathcal{V}_i|^2 \cdot d)$, where d represents the feature dimension. After obtaining these embeddings, we combine them to form the final graph embeddings $\mathbf{H}$. Then, we construct distinct GoGs G_{topo} and G_{fea} and perform GoG propagation, with a time complexity of $\mathcal{O}(k^2 \cdot (|\mathcal{E}_{topo}| + |\mathcal{E}_{fea}|))$, where k is the number of graphs, and $|\mathcal{E}_{topo}|$ and $|\mathcal{E}_{fea}|$ represent the number of edges in G_{topo} and G_{fea} respectively. Subsequently, we average the graph representations after the two GoG propagations to obtain the final graph representation $\mathbf{Z}$, and calculate multiple loss functions through Eqs. (9)–(11), which guide the optimization of model parameters through backpropagation. Specifically, the total loss $\mathcal{L}_{total}$ is defined as the sum of the approximation loss $\mathcal{L}_{app}$, the single-graph reconstruction loss $\mathcal{L}_{rec}^{sin}$, and the graph-of-graph reconstruction loss $\mathcal{L}_{rec}^{gog}$, expressed as:

$$\mathcal{L}_{total} = \mathcal{L}_{app} + \mathcal{L}_{rec}^{sin} + \mathcal{L}_{rec}^{gog}. \tag{14}$$

In the inference phase, we follow similar steps to calculate embeddings and construct GoGs. Specifically, we calculate GNN-based and Transformer-based embeddings for each graph, obtain combined graph embeddings, and construct two different GoGs. After propagating through the GoGs, we average the graph representations to get the final embeddings. Finally, we calculate the anomaly scores for each graph. This involves computing both intra-graph and inter-graph anomaly scores and then combining them to produce the final anomaly score. The final output of the algorithm is an anomaly score ranking list for all the graphs.

4. Experiments

In this section, we perform extensive experiments on eight real-world datasets to showcase the effectiveness of our proposed GoGRL method. We begin by introducing the datasets used and outlining the experimental setup, which includes the baseline methods for comparison, implementation details, and evaluation metrics. Next, we address the following research questions:

- **RQ1:** How does GoGRL compare to state-of-the-art baselines across various real-world datasets?
- **RQ2:** How does GoGRL perform in terms of time efficiency?
- **RQ3:** What impact do different components have on the performance of GoGRL?
- **RQ4:** How do various hyperparameter settings influence model performance?

Table 2
The statistics of the real-world networks.

	Dataset	#Graphs	#Avg.Nodes	#Avg.Edges	#Anomaly rate
Biochemical molecule	PROTEINS	1113	39.06	72.82	0.4
	ENZYMES	600	32.63	62.14	0.17
	AIDS	2000	15.69	16.2	0.2
	DHFR	467	42.43	44.54	0.39
	BZR	405	35.75	38.36	0.21
	COX2	467	41.22	43.45	0.22
Social network	IMDB	1000	19.77	96.53	0.50
	REDDIT	2000	429.63	497.75	0.50

- **RQ5:** What types of graph representations and anomaly score distributions are produced by GoGRL?

4.1. Datasets

To validate the effectiveness of GoGRL, we conduct experiments on eight publicly available graph datasets from two popular application domains. These include six biochemical molecule datasets (PROTEINS, ENZYMES, AIDS, DHFR, BZR, COX2) and two social network datasets (IMDB and REDDIT). Their statistical information is tabulated in Table 2.

- **PROTEINS** (Borgwardt et al., 2005) is a collection of graphs where each graph represents a protein. Nodes correspond to secondary structure elements such as helices and sheets, while edges denote spatial proximity between these elements. This dataset is used to classify proteins into enzymes and non-enzymes, helping in understanding the structure-function relationship in proteins.
- **ENZYMES** (Borgwardt et al., 2005) contains graphs representing enzyme molecules. Each graph node represents a secondary structure element, and edges indicate spatial neighborhood relationships. The classification task involves categorizing each graph into one of six enzyme classes according to their Enzyme Commission (EC) numbers.
- **AIDS** (Riesen & Bunke, 2008) is composed of molecular graphs where nodes represent atoms, and edges represent chemical bonds. The dataset is utilized to classify compounds based on their ability to inhibit the HIV virus, aiding in antiviral drug research.
- **DHFR** (Sutherland et al., 2003) includes molecular graphs used to classify molecules based on their inhibitory activity against DHFR, an enzyme critical for DNA synthesis. This dataset is crucial for developing anticancer and antimicrobial drugs.
- **BZR** (Sutherland et al., 2003) consists of molecular graphs where each graph represents a molecule that potentially binds to the benzodiazepine receptor. The classification task involves predicting the binding affinity of these molecules, which is essential for developing therapeutic agents.
- **COX2** (Sutherland et al., 2003) contains molecular graphs aimed at predicting the inhibitory activity of molecules against the COX2 enzyme, which plays a significant role in inflammation and pain. This dataset is critical for the development of anti-inflammatory drugs.
- **IMDB** (Yanardag & Vishwanathan, 2015) consists of ego-networks of actors and actresses, where each node represents an actor or actress and edges denote co-acting in the same movie. The classification task is to categorize each graph based on the genre of the movies the actors have worked in. This dataset contains 1500 graphs, providing insights into social structures within the film industry.
- **REDDIT** (Yanardag & Vishwanathan, 2015) comprises user interaction graphs from various subreddits. Nodes represent users, and edges denote interactions such as comments and replies. The classification task involves categorizing each graph based on the topic of the subreddit. This dataset contains 2000 graphs, capturing the dynamic and diverse nature of online communities.

Since there are no ground-truth anomalies in the aforementioned datasets, we follow the methodology employed in previous research (Li et al., 2023a; Ma et al., 2022; Niu et al., 2023) to adapt these datasets for evaluating our model in the GLAD task. Specifically, we identified graphs from minority classes in these datasets as anomalous samples. Subsequently, we merged graphs from the remaining classes in each dataset and labeled them as normal samples. This approach adheres to the foundational assumption in anomaly detection tasks that less prevalent minority class samples can serve as anomalies. Thus, through this strategy, we simulated a realistic scenario where anomalies are rare and interspersed among a large number of normal instances. This setup facilitates a thorough evaluation of our model's ability to accurately detect anomalies, ensuring its robustness and reliability in practical anomaly detection applications.

4.2. Experimental settings

4.2.1. Baselines methods

To evaluate the performance, we compared GoGRL with two groups of methods: two-stage detection methods and end-to-end detection methods. For each group, we select representative methods and briefly introduce them below.

Group 1: Two-stage methods. Following previous research, we adopt the Propagation Kernel (PK) (Neumann et al., 2016) and Weisfeiler–Lehman Kernel (WL) (Shervashidze et al., 2011) as representation learners to acquire graph-level embeddings in two-stage methods. We employed Isolation Forest (iF) (Liu et al., 2008) and One-Class SVM (OCSVM) (Manevitz & Yousef, 2001) as anomaly detectors for detecting graph anomalies within the two-stage framework. Subsequently, we combined these components to create the following four two-stage GLAD methods for comparison:

- **PK-OCSVM** utilizes the PK graph kernel to generate graph embeddings by capturing structural information through propagation techniques. Subsequently, anomalies are identified based on these embeddings using the One-Class Support Vector Machine.
- **PK-iF** leverages the PK graph kernel to derive graph embeddings that emphasize propagation-based structural features. Anomalies are detected using these embeddings through the Isolation Forest algorithm.
- **WL-OCSVM** utilizes the Weisfeiler–Leman kernel to derive graph embeddings that focus on detailed structural patterns refined through iterative label aggregation. Anomalies are subsequently identified based on these embeddings using the One-Class Support Vector Machine.
- **WL-iF** employs the Weisfeiler–Leman kernel to generate graph embeddings that encapsulate intricate structural information derived from iterative label refinement. These embeddings are subsequently used by the Isolation Forest algorithm to detect anomalies.

Group 2: End-to-end methods. With the latest advancements in graph anomaly detection, various end-to-end methods integrate graph

representation learning and anomaly detection into a single framework. These methods utilize different neural network architectures and mechanisms to directly learn embeddings that highlight anomalous patterns in the graph data. For comparison, we select the following representative end-to-end methods:

- **GLocalKD** (Ma et al., 2022) adopts GNNs as encoders and implements random knowledge distillation.
- **OCGIN** (Zhao & Akoglu, 2023) utilizes GIN as the graph encoder and employs average pooling to generate graph representations, while optimizing the model with a distance to center point and regularization term.
- **GOOD-D** (Liu et al., 2023) introduces a novel non-perturbative graph data augmentation technique and employs GIN as an encoder for graph-level anomaly detection.
- **HimNet** (Niu et al., 2023) introduces a memory network to separately optimize different level representations from a global perspective, resulting in more discriminative graph-level representations.
- **SIGNET** (Liu et al., 2024) introduces dual hypergraph and extracts bottleneck subgraphs under different views based on information bottleneck theory. The mutual information between the two subgraphs is used to generate anomaly score of each graph.

4.2.2. Evaluation metrics

To evaluate the performance of our proposed model alongside the baseline models, we employ ROC-AUC as the evaluation metric. This metric is widely used in prior studies (Liu et al., 2023; Ma et al., 2022; Niu et al., 2023) to measure the effectiveness of anomaly detection methods. The ROC curve itself is a graphical representation that plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings. The AUC-ROC score quantifies the overall ability of the model to distinguish between positive and negative classes by measuring the area under this curve. A higher AUC-ROC value, which ranges from 0 to 1, indicates a better performance of the model in distinguishing between the two classes. Specifically, an AU-ROC value of 1 signifies perfect classification, while a value of 0.5 denotes a model with no discriminative power, akin to random guessing.

4.2.3. Implementation details

For the baseline methods mentioned, we use the open-source implementations provided by the authors for OCCGIN,[1] GLocalKD,[2] GOOD-D,[3] HimNet,[4] and SIGNET.[5] Our experiments with these methods adhere to the optimal hyperparameter configurations outlined in their respective papers.

For our GoGRL, we use PyTorch for implementation and train the model with the Adam optimizer, setting the learning rate α to $1e^{-3}$ for both the GNN encoders and the graph transformer. Additionally, for datasets with a larger number of samples (e.g., PROTEINS, AIDS, IMDB, and REDDIT) and other smaller datasets, the dimensions of the GNN encoders d_{gnn} and graph transformers d_{tra} are set to 64, 128 and 32, 128, respectively, while the controlling coefficient λ is fixed at 0.5. The structural encoding dimensions d_{se} are set to 16 for biochemical molecular networks and 32 for social networks. When constructing the graph-of-graphs, the number of neighbors k is considered to be 3 or 4. All experiments are carried out on NVIDIA RTX 4090 24 GB GPU with Intel Xeon(R) Platinum CPU. Furthermore, to enhance clarity, we provide a detailed summary of the key hyperparameter settings for different datasets in Table 3.

[1] https://github.com/LingxiaoShawn/GLOD-Issues
[2] https://git.io/GLocalKD
[3] https://github.com/yixinliu233/G-OOD-D
[4] https://github.com/Niuchx/HimNet
[5] https://github.com/yixinliu233/SIGNET

Table 3
Hyperparameter settings for GoGRL on different datasets..

Dataset	d_{gnn}	d_{tra}	d_{se}	k	λ	α	epochs
PROTEINS	64	128	16	4	0.5	$1e^{-3}$	200
ENZYMES	32	128	16	3	0.5	$1e^{-3}$	150
AIDS	64	128	16	4	0.5	$1e^{-3}$	150
DHFR	32	128	16	3	0.5	$1e^{-3}$	150
BZR	32	128	16	3	0.5	$1e^{-3}$	150
COX2	32	128	16	3	0.5	$1e^{-3}$	150
IMDB	64	128	32	4	0.5	$1e^{-3}$	200
REDDIT	64	128	32	4	0.7	$1e^{-3}$	300

4.3. Performance comparison (RQ1)

To address RQ1, we conduct five tests on our method and all other methods across eight datasets, reporting the average and standard deviation of AUC results. Tables 4 and 5 respectively present the detailed performance comparisons of our GoGRL with two-stage detection methods and end-to-end detection methods. According to the results, we have the following observations:

- Compared to end-to-end deep learning methods, two-stage methods exhibit poorer performance across all datasets. This outcome may be attributed to several reasons: Firstly, in our experimental setup, we selected two commonly used representation learners and anomaly detectors independently and combined them. In such a setup, we cannot guarantee that we have identified the optimal combination for achieving the best anomaly detection performance; there may be superior combinations of two-stage methods yet to be explored. Secondly, since representation learners and anomaly detectors in two-stage methods are often trained separately, the lack of a unified training objective between them might contribute to the lower anomaly detection performance. Additionally, traditional machine learning methods as detectors often rely on real labels for training, potentially leading to overfitting of normal labels in anomaly detection problems. In real-world anomaly detection scenarios, normal samples typically outnumber abnormal ones by far.
- For end-to-end deep learning methods, approaches that consider inter-graph relationships modeling (such as HimNet) often exhibit better performance compared to other deep learning methods that lack such inter-graph relationship modeling. A reasonable explanation for this is that most methods treat each graph sample as an independent entity and view graph anomaly detection as a simple binary classification task. Such an assumption overlooks the potential complex anomaly patterns between graphs, whereas modeling inter-graph relationships helps capture these patterns, resulting in better performance.
- Our method achieve the best performance on most datasets, highlighting the intuitive importance of simultaneously considering and thoroughly exploring inter-graph and intra-graph relationships for the GLAD task. Specifically, we supplemented the GNN encoder with graph transformers, addressing the deficiency in exploring intra-graph anomaly parts due to limited receptive fields. Our proposed Transformer-enhanced graph encoder effectively explores potential anomaly regions within each graph. Additionally, the graph-of-graph propagation mechanism, constructed from both structural and attribute perspectives, comprehensively and explicitly models inter-sample relationships across graphs. This approach transforms individual graph samples from independent entities into interconnected entities, aiding in the exploration of potential inter-graph anomaly patterns.

Table 4
The AUC results (expressed as percentages, with mean ±standard deviation) for both the two-stage detection methods and our proposed approach across eight real-world graph datasets. The top performance in each row is highlighted in bold, while the second-best performance is underlined.

Method	PK-OCSVM	PK-iF	WL-OCSVM	WL-iF	GoGRL
PROTEINS	50.49 ± 4.92	60.70 ± 2.55	51.35 ± 4.35	61.36 ± 2.54	**78.63 ± 4.71**
ENZYMES	53.67 ± 2.66	51.30 ± 2.01	55.24 ±2.66	51.60 ± 3.81	**64.04 ± 1.61**
AIDS	50.79 ± 4.30	51.84 ± 2.87	50.12 ± 3.43	61.13 ± 0.71	**98.69 ± 2.25**
DHFR	47.91 ± 3.76	52.11 ± 3.96	50.24 ± 3.13	50.29 ± 2.77	**74.77 ± 5.63**
BZR	46.85 ± 5.31	55.32 ± 6.18	50.56 ± 5.87	52.46 ± 3.30	**82.65 ± 4.89**
COX2	50.27 ± 7.91	50.05 ± 2.06	49.86 ± 7.43	50.27 ± 0.34	**72.11 ± 6.21**
IMDB	50.75 ± 3.10	50.80 ± 3.17	54.08 ± 5.19	50.20 ± 0.40	**69.64 ± 6.42**
REDDIT	45.68 ± 2.24	46.72 ± 3.42	49.31 ± 2.33	48.26 ± 0.32	**84.56 ± 5.93**

Table 5
The AUC results (expressed as percentages, with mean ±standard deviation) for both the end-to-end detection methods and our proposed approach across eight real-world graph datasets. The top performance in each row is highlighted in bold, while the second-best performance is underlined.

Method	OCGIN	GLocalKD	GOOD-D	HimNET	SIGNET	GoGRL
PROTEINS	70.89 ± 2.44	77.30 ± 5.15	71.97 ± 3.86	77.20 ± 1.5	75.22 ± 3.91	**78.63 ± 4.71**
ENZYMES	58.75 ± 5.98	61.39 ± 8.81	63.90 ± 3.69	58.9 ±7.6	62.96 ± 4.22	**64.04 ± 1.61**
AIDS	78.16 ± 3.05	93.27 ± 4.19	97.28 ± 0.69	**99.7 ±0.3**	97.27 ± 1.17	98.69 ± 2.25
DHFR	49.23 ± 3.05	56.71 ± 3.57	62.67 ± 3.11	70.1 ± 1.7	74.01 ± 4.69	**74.77 ± 5.63**
BZR	65.91 ± 1.47	69.42 ± 7.78	75.16 ±5.15	70.3 ± 5.4	81.44 ± 9.23	**82.65 ± 4.89**
COX2	53.58 ± 5.05	59.37 ± 12.67	62.65 ± 8.14	63.7 ± 7.6	71.46 ± 4.64	**72.11 ± 6.21**
IMDB	60.19 ± 8.90	52.09 ± 3.41	65.88 ± 0.75	68.3 ± 3.2	66.48 ± 3.49	**69.64 ±6.42**
REDDIT	75.93 ± 8.65	77.85 ± 2.62	**88.67 ± 1.24**	78.0 ±2.5	82.78 ± 1.11	84.56 ± 5.93

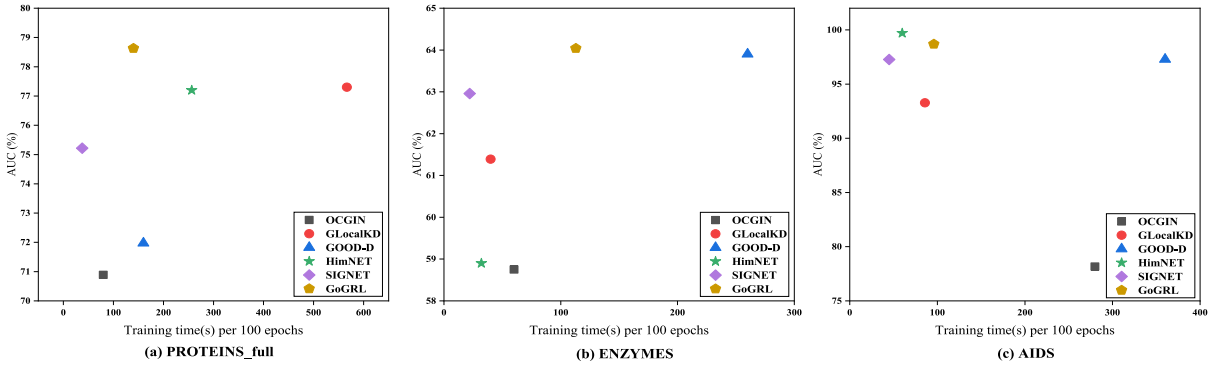


Fig. 3. Model comparison of the time efficiency on three datasets.

4.4. Efficiency analysis (RQ2)

To address RQ2, this subsection presents a comparative analysis of the runtime and anomaly detection performance of our model against five state-of-the-art end-to-end GLAD models (i.e., OCGIN, GLocalKD, GOOD-D, HimNet, and SIGNET), all evaluated under the same hardware conditions. Fig. 3 provides scatter plots of the experimental results across three bioinformatics datasets of varying scales, with the y-axis representing the AUC of anomaly detection and the x-axis depicting the runtime per 100 epochs, given that each model achieves its optimal performance at different epochs. Models positioned closer to the top-left corner exhibit superior performance.

As illustrated in Fig. 3, our proposed method consistently achieves either the best or second-best performance across all datasets while maintaining an acceptable runtime, outperforming other cutting-edge baseline models. From a performance perspective, the introduction of a Transformer-enhanced graph encoder and dual-view graph-of-graph representation learning effectively captures information both within and between graphs, producing discriminative representations that are highly beneficial for GLAD tasks. In terms of efficiency, although we previously noted the high computational complexity associated with constructing kernel functions, this is a one-time preprocessing step, ensuring that our model's overall runtime remains within an acceptable range.

4.5. Ablation study (RQ3)

To address RQ3, we conduct experiments using real datasets to evaluate our proposed GoGRL method and its three ablation variants. The differences between the ablation variants and our proposed method are summarized in Table 6. The purpose of this study is to thoroughly understand whether each component of GoGRL effectively explores intra-graph and inter-graph anomaly information, thereby influencing the performance of graph-level anomaly detection. The results of the ablation experiments are shown in Fig. 4.

Effect of transformer-enhanced embedding. We first evaluate the effectiveness of introducing the Transformer-enhanced graph encoder in generating more informative intra-graph node representations and improving the performance of graph-level anomaly detection. To this end, we remove the graph transformer from the proposed Transformer-enhanced graph encoder to create a model variant GoGRL(w/o)-TGE. The results presented in Fig. 4 indicate that GoGRL(w/o)-TGE exhibits a certain performance decline on almost all datasets compared to the original GoGRL. This result fully demonstrates that the Graph Transformer introduced in our proposed method effectively expands the receptive field of traditional GNNs to capture latent anomalous patterns within individual graphs, generating discriminative graph representations, and thereby enhancing anomaly detection performance.

Effect of different Graph-of-Graph views. To investigate whether our explicit modeling of the two GoG perspectives can effectively

Table 6
Comparison of GoGRL and its ablation variants.

Model	Graph transformer	Topology-based GoG	Feature-based GoG
GoGRL(w/o)-TGE	–	✓	✓
GoGRL(w/o)-TOPO	✓	–	✓
GoGRL(w/o)-FEA	✓	✓	–
GoGRL (ours)	✓	✓	✓

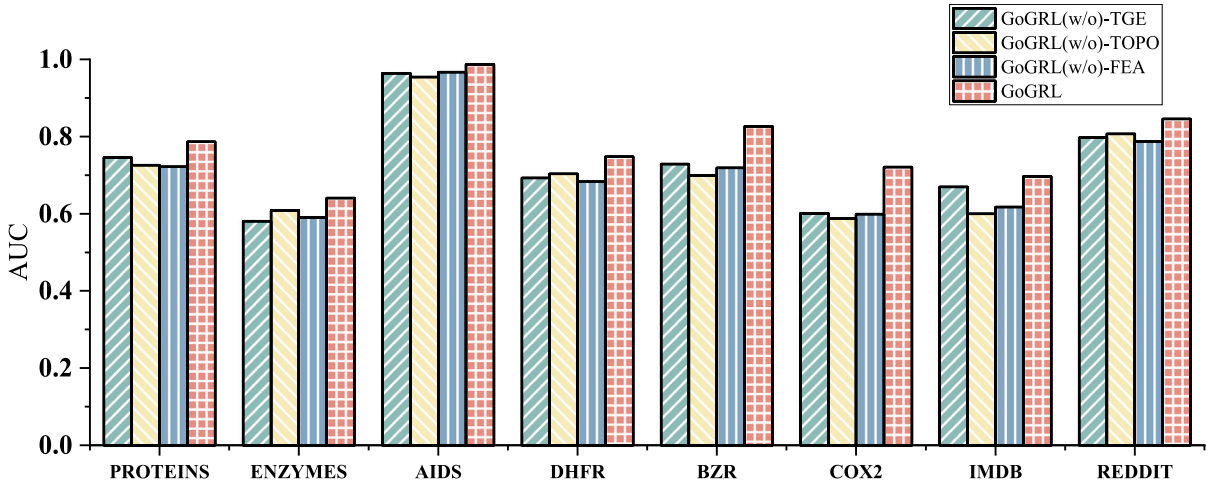


Fig. 4. Ablation study results of GoGRL and its variants.

explore inter-graph anomaly information, we propose two variants: GoGRL(w/o)-TOPO and GoGRL(w/o)-FEA. The former removes the topology-based graph-in-graph representation learning from the dual-scale graph-in-graph approach, while the latter removes the attribute-based graph-in-graph representation learning. The results presented in Fig. 4 show that, compared to GoGRL with both scales of the graph-in-graph perspectives, reducing either perspective results in a decline in the model's anomaly detection performance. This result indicates that our proposed explicit modeling strategy for inter-graph relationships comprehensively and effectively explores the anomalous patterns inherent in inter-graph relationships, providing clearer insights for graph-level anomaly detection.

4.6. Hyperparameter analysis (RQ4)

In this subsection, we examine the impact of different settings of several key hyperparameters in GoGRL, including neighborhood numbers k, controlling coefficient λ and detection threshold N.

Effect of neighborhood numbers k. To examine how the number of graph-of-graph neighbors k affects model performance, we conduct experiments with k ranging from 1 to 8. The results, depicted in Fig. 5(a), reveal a clear trend: initially, as k increases, the model's performance improves across all datasets. This improvement is due to the effectiveness of our GoG construction method in identifying similar neighbors for each graph sample, with more similar neighbors providing additional useful information for graph learning. However, performance begins to deteriorate with further increases in k, particularly when k reaches 7. This decline suggests that a very large k may lead to the selection of neighbors that are less relevant to the sample, introducing noise that negatively impacts anomaly detection.

Impact of controlling coefficient λ. The controlling coefficient λ is vital in the anomaly scoring function as it adjusts the weight given to intra-graph and inter-graph anomaly scores. As illustrated in Fig. 5(b), increasing λ generally enhances model performance. Notably, the model performs best on most datasets when λ is set to 0.5 or 0.7, after which performance tends to decline slightly. This suggests that the model achieves an optimal balance between detecting intra-graph and inter-graph anomalies. This balance is crucial for the GLAD task because, as hypothesized, the anomaly status of a graph is inherently connected to both its own properties and its relationships with other graphs. Thus, a model that effectively addresses both types of anomalies is more robust.

Effect of detection threshold N. The detection threshold N plays a crucial role in our method, as it directly determines which graph samples in the descending order of anomaly scores are classified as anomalous. More specifically, N represents a proportion, indicating the top-N% of samples in the sorted list that will be labeled as anomalous. To evaluate the effect of different N on model performance, we conduct experiments by changing N from 10% to 50%. As shown in Fig. 5(c), by synthesizing the results across all datasets, we observe that, for each dataset, the model tends to achieve better performance when the value of N is close to the inherent anomaly rate of the dataset. When N is either too large or too small, model performance suffers, possibly due to misclassifying a large number of normal samples as anomalous, or failing to detect many anomalous samples, mislabeling them as normal. Therefore, selecting an appropriate value for N based on the specific context to obtain robust detection results is a key consideration in the design of our model.

4.7. Visualization (RQ5)

In this subsection, we aim to evaluate how well the GoGRL model captures anomalous features from both intra-graph and inter-graph perspectives and assess the overall effectiveness of the designed anomaly scoring strategy. To accomplish this, we visualize the initial graph embeddings, the embeddings learned by each component of our proposed model, and the resulting anomaly scores. The visualization results and the distribution of anomaly scores are presented in Figs. 6 and 7.

Visual embedding. To gain a more intuitive understanding of how effectively the GoGRL model learns graph-level representations that extract anomalous information, we use t-SNE (Van der Maaten & Hinton, 2008) to map the high-dimensional embeddings into a two-dimensional space. As shown in Fig. 6(a), (b), and (c), they respectively display the visualization of graph-level representations derived from the original graph, the Transformer-enhanced strategy, and GoG propagation, where each point represents a graph from the test set.

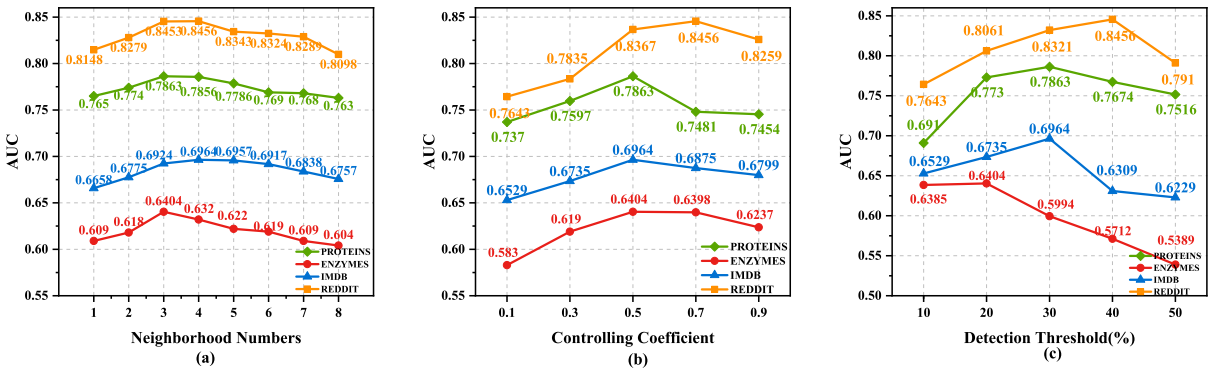


Fig. 5. Hyperparameter analysis results of GoGRL on two biomolecular networks and two social networks..

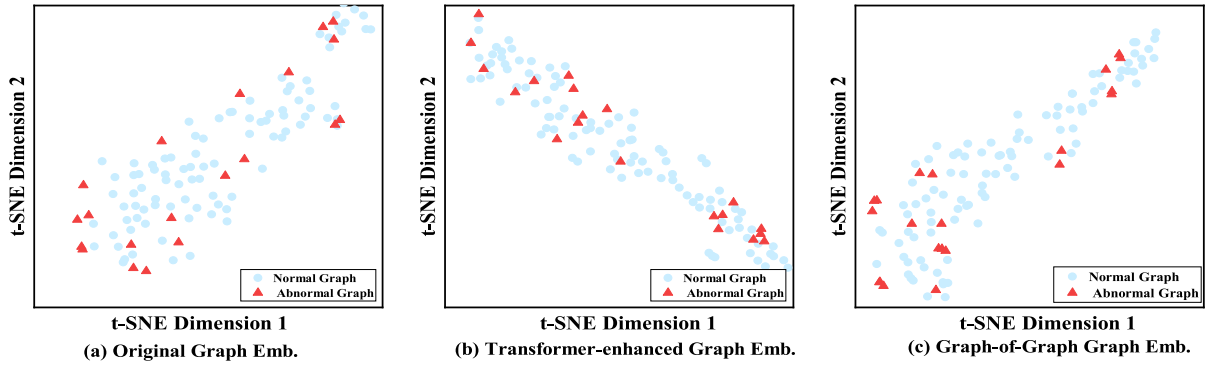


Fig. 6. Visualization on test graph samples from ENZYMES dataset. (a) t-SNE visualization of original graph embeddings (emb.). (b) t-SNE visualization of Transformer-enhanced graph embeddings (emb.). (c) t-SNE visualization of Graph-of-Graph graph embeddings (emb.).

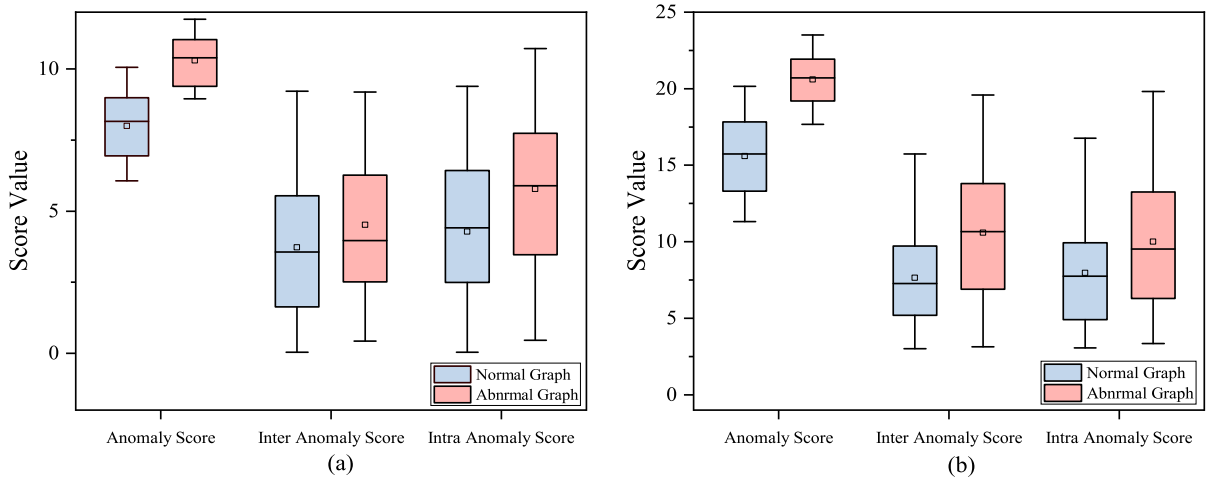


Fig. 7. Distribution of anomaly score of test samples on ENZYMES and AIDS. (a) Box plots of intra-graph anomaly score, inter-graph anomaly score, and overall anomaly score on the ENZYMES dataset. (b) Box plots of intra-graph anomaly score, inter-graph anomaly score, and overall anomaly score on the AIDS dataset.

From Fig. 6, we have the following key observations: In Fig. 6(a), we observe that the anomalous and normal graph samples do not exhibit clear decision boundaries, and their embeddings are relatively scattered, making it difficult to discern distinct clustering patterns. However, in Fig. 6(b), we notice that some anomalous graph samples begin to form clusters. Overall, both normal and anomalous samples have embeddings that are closer to those of the same class, indicating that our proposed Transformer-enhanced graph encoder module effectively captures hidden anomalous information within the graphs, which the initial graph embeddings lack. Finally, in Fig. 6(c), we observe more pronounced clustering of anomalous graph samples, which can be attributed to our proposed dual-channel graph-of-graph representation learning module effectively extracting inter-sample information. This module provides richer inter-graph information for the representation learning process. In summary, the intra-graph and inter-graph information exploration strategies included in our approach progressively extract latent anomalous information from graph samples, generating informative graph-level embeddings. This provides important insights for the GLAD task.

Anomaly score distribution. To provide a more intuitive demonstration of the underlying principles and effectiveness of the anomaly

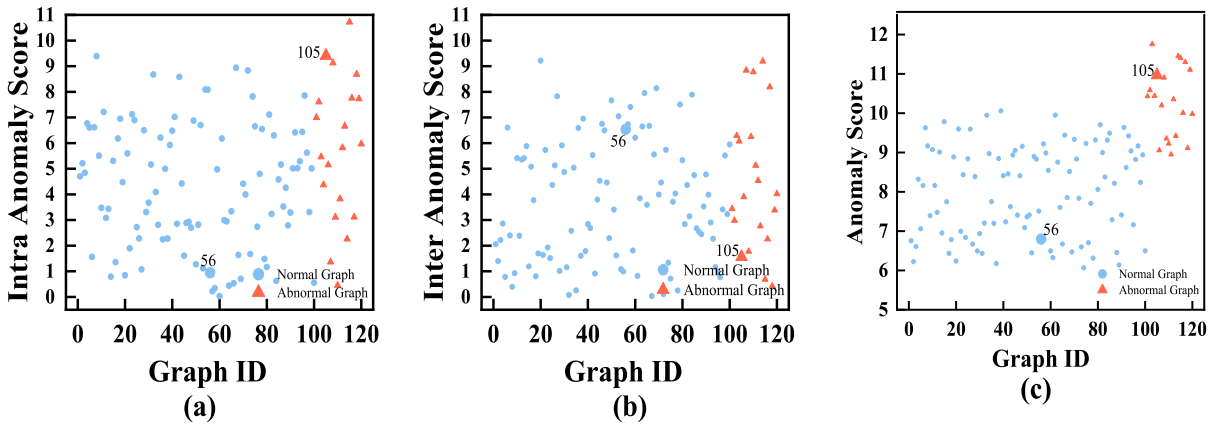


Fig. 8. Scatterplot visualization of intra-graph anomaly score, inter-graph anomaly score, and overall anomaly score for all test samples in the ENZYMES dataset.

score design in the GoGRL model, we conduct a comprehensive statistical analysis of the anomaly scores for each test instance within the ENZYMES and AIDS datasets, as depicted in Fig. 7. The box plots in the figure depict the distribution of intra-graph, inter-graph, and overall anomaly scores for different classes across both datasets. In the plots, red and blue are used to represent anomalous and normal graph samples, respectively, while the squares within the box plots indicate the mean values of the scores for each class.

From the visualization in Fig. 7, two key observations emerge: **Obs1:** Within the same dataset, the distributions of intra-graph and inter-graph anomaly scores for each sample are distinct, indicating that no correlation exists between these two types of scores. This underscores the importance of integrating both intra-graph and inter-graph anomaly scores in the design of the overall anomaly score. **Obs2:** Across different datasets, due to variations in the structure and attributes of graph samples, observing only the intra-graph or inter-graph anomaly scores does not reveal consistent distribution patterns. However, when examining the overall anomaly scores, a similar distribution pattern is evident in both the ENZYMES and AIDS datasets: anomalous samples consistently exhibit higher scores compared to normal samples. This finding supports our hypothesis regarding the anomaly score design, as the core principle of reconstruction-based anomaly detection is that anomalous samples are intrinsically more challenging to reconstruct than normal ones.

Additionally, to further clarify and intuitively illustrate the rationale behind the design of the two types of anomaly scores in our method, we conduct a scatterplot visualization of the three anomaly scores for all test samples in the ENZYMES dataset, as shown in Fig. 8. Each point in the figure represents a test sample, with red and blue indicating anomalous and normal samples, respectively. From Fig. 8, we observe that when considering only a single type of anomaly score, the scores may exhibit poor discriminative power. For example, anomalous samples may have lower scores, or normal samples may have higher scores, leading to inaccurate detection results. Specifically, we select two graph samples with IDs 56 and 105 to illustrate this phenomenon. In Fig. 8(b), the normal graph (ID = 56) has a high inter-graph anomaly score, while the anomalous graph (ID = 105) has a low inter-graph anomaly score. Relying solely on this score for detection may lead to inaccurate detection results. Nonetheless, in Fig. 8(c), the alignment of scores with our hypothesis is evident when both intra-graph and inter-graph anomaly scores are amalgamated. Normal samples tend to yield lower scores, whereas anomalous samples tend to yield higher scores. This shift strongly underscores the significance of cohesively structuring anomaly scores from both intra-graph and inter-graph viewpoints in our methodology.

5. Conclusion

Fully addressing the challenge of inadequate exploration of intra-graph and inter-graph messages in existing research is crucial for achieving high-quality graph-level anomaly detection. In this paper, we propose a dual-view graph-of-graph representation learning method combined with a graph transformer for unsupervised graph-level anomaly detection, which thoroughly explores the information contained in graph samples from both intra-graph and inter-graph perspectives. Specifically, our framework incorporates a transformer-enhanced graph encoder, leveraging the advantages of self-attention mechanisms to extend the receptive field of GNNs and achieve more comprehensive intra-graph information mining. Furthermore, the dual-view graph-of-graph representation learning network constructs two graphs within the graph from different perspectives, facilitating effective exploration of cross-graph information. Experimental results demonstrate the effectiveness of our proposed framework for the GLAD task across a wide range of benchmark datasets compared to baseline methods.

CRediT authorship contribution statement

Wangyu Jin: Writing – review & editing, Writing – original draft, Visualization, Software, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Huifang Ma:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition, Conceptualization. **Yingyue Zhang:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization. **Zhixin Li:** Supervision, Funding acquisition. **Liang Chang:** Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study is supported by National Natural Science Foundation of China (62441701 and U22A2099), Gansu Province Key Foundation Project, China (No. 24JRRA123), the Industrial Support Project of Gansu Colleges, China (No. 2022CYZC11), and the Graduate Research Support Project of Northwest Normal University, China (2023KYZZ-S133).

Data availability

The datasets used in this paper are all available from open source.

References

Borgwardt, K. M., & Kriegel, H.-P. (2005). Shortest-path kernels on graphs. In *Fifth IEEE international conference on data mining* (pp. 8–pp). IEEE.

Borgwardt, K. M., Ong, C. S., Schönauer, S., Vishwanathan, S., Smola, A. J., & Kriegel, H.-P. (2005). Protein function prediction via graph kernels. *Bioinformatics*, *21*, i47–i56.

Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD international conference on management of data* (pp. 93–104).

Chai, Z., Yang, Y., Dan, J., Tian, S., Meng, C., Wang, W., & Sun, Y. (2023). Towards learning to discover money laundering sub-network in massive transaction network. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 14153–14160).

Dwivedi, V. P., & Bresson, X. (2021). A generalization of transformer networks to graphs. In *Proceedings of the AAAI conference on artificial intelligence*.

Dwivedi, V. P., Luu, A. T., Laurent, T., Bengio, Y., & Bresson, X. (2022). Graph neural networks with learnable structural and positional representations. In *Proceedings of the 10th international conference on learning representations*.

Gao, Z., Ma, H., Zhang, X., Wang, Y., & Wu, Z. (2023). Similarity measures-based graph co-contrastive learning for drug–disease association prediction. *Bioinformatics*, *39*, btad357.

Jiang, Y., Ma, H., Zhang, X., LI, Z., & Chang, L. (2024). Incorporating metapath interaction on heterogeneous information network for social recommendation. *Frontiers of Computer Science*, *18*, Article 181302.

Jin, W., Ma, H., Zhang, Y., Li, Z., & Chang, L. (2024). Multi-view discriminative edge heterophily contrastive learning network for attributed graph anomaly detection. *Expert Systems with Applications*, Article 124460.

Kreuzer, D., Beaini, D., Hamilton, W., Létourneau, V., & Tossou, P. (2021). Rethinking graph transformers with spectral attention. *Advances in Neural Information Processing Systems*, *34*, 21618–21629.

Li, J., Xing, Q., Wang, Q., & Chang, Y. (2023). CVTGAD: Simplified transformer with cross-view attention for unsupervised graph-level anomaly detection. In *Joint European conference on machine learning and knowledge discovery in databases* (pp. 185–200). Springer.

Li, P., Yu, H., Luo, X., & Wu, J. (2023). LGM-GNN: A local and global aware memory-based graph neural network for fraud detection. *IEEE Transactions on Big Data*, *9*, 1116–1127.

Liu, Z., Chen, C., Yang, X., Zhou, J., Li, X., & Song, L. (2018). Heterogeneous graph neural networks for malicious account detection. In *Proceedings of the 27th ACM international conference on information and knowledge management* (pp. 2077–2085).

Liu, Y., Ding, K., Liu, H., & Pan, S. (2023). Good-d: On unsupervised graph out-of-distribution detection. In *Proceedings of the sixteenth ACM international conference on web search and data mining* (pp. 339–347).

Liu, Y., Ding, K., Lu, Q., Li, F., Zhang, L. Y., & Pan, S. (2024). Towards self-interpretable graph-level anomaly detection. *Advances in Neural Information Processing Systems*, *36*.

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *2008 eighth IEEE international conference on data mining* (pp. 413–422).

Ma, R., Pang, G., Chen, L., & van den Hengel, A. (2022). Deep graph-level anomaly detection by glocal knowledge distillation. In *Proceedings of the fifteenth ACM international conference on web search and data mining* (pp. 704–714).

Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-sne.. *Journal of Machine Learning Research*, *9*.

Manevitz, L. M., & Yousef, M. (2001). One-class SVMs for document classification. *Journal of Machine Learning Research*, *2*, 139–154.

Mialon, G., Chen, D., Selosse, M., & Mairal, J. (2021). Graphit: Encoding graph structure in transformers. arXiv preprint arXiv:2106.05667.

Neumann, M., Garnett, R., Bauckhage, C., & Kersting, K. (2016). Propagation kernels: efficient graph kernels from propagated information. *Machine Learning*, *102*, 209–245.

Niu, C., Pang, G., & Chen, L. (2023). Graph-level anomaly detection via hierarchical memory networks. In *Joint European conference on machine learning and knowledge discovery in databases* (pp. 201–218). Springer.

Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., & Beaini, D. (2022). Recipe for a general, powerful, scalable graph transformer. *Advances in Neural Information Processing Systems*, *35*, 14501–14515.

Riesen, K., & Bunke, H. (2008). IAM graph database repository for graph based pattern recognition and machine learning. In *Structural, syntactic, and statistical pattern recognition: joint IAPR international workshop, SSPR & SPR 2008, orlando, USA, December 4-6, 2008. proceedings* (pp. 287–297).

Shaw, P., Uszkoreit, J., & Vaswani, A. (2018). Self-attention with relative position representations. In *Proceedings of the 2018 conference of the North American chapter of the association for computational linguistics*.

Shervashidze, N., Schweitzer, P., Van Leeuwen, E. J., Mehlhorn, K., & Borgwardt, K. M. (2011). Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, *12*.

Sutherland, J. J., O'brien, L. A., & Weaver, D. F. (2003). Spline-fitting with a genetic algorithm: A method for developing classification structure- activity relationships. *Journal of Chemical Information and Computer Sciences*, *43*, 1906–1915.

Wang, Y., Min, Y., Chen, X., & Wu, J. (2021). Multi-view graph contrastive representation learning for drug-drug interaction prediction. In *Proceedings of the web conference 2021* (pp. 2921–2933).

Wu, Z., Ma, H., Deng, B., Li, Z., & Chang, L. (2024). Co-guided dual-channel graph neural networks for the prediction of compound-protein interaction. *Applied Soft Computing*, Article 111875.

Wu, Q., Zhao, W., Li, Z., Wipf, D. P., & Yan, J. (2022). Nodeformer: A scalable graph structure learning transformer for node classification. *Advances in Neural Information Processing Systems*, *35*, 27387–27401.

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? In *Proceedings of international conference on learning representations 2019*.

Xu, J., Yu, S., Sun, K., Ren, J., Lee, I., Pan, S., & Xia, F. (2020). Multivariate relations aggregation learning in social networks. In *Proceedings of the ACM/IEEE joint conference on digital libraries in 2020* (pp. 77–86).

Yanardag, P., & Vishwanathan, S. (2015). Deep graph kernels. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1365–1374).

Zhao, L., & Akoglu, L. (2023). On using classification datasets to evaluate graph outlier detection: Peculiar observations and new insights. *Big Data*, *11*, 151–180.